Système d'exploitation
CM1

Pierre Rousselin
rousselin@univ-paris13.fr

Université Sorbonne Paris Nord
L2 informatique et double-licence

janvier 2026

Noyau et appels système

Processus Unix

Rappels de C : `switch`

Processus : naissance et mort (1)

Noyau de système d'exploitation

Le noyau d'un système d'exploitation (*kernel*) est

- un composant logiciel (un programme),
- chargé en mémoire au démarrage de la machine,
- qui doit gérer les autres logiciels (ordonnancement, lancement de tâches, communication entre processus, ...),
- et les interactions avec le matériel (entrées/sorties, périphériques de stockage, mémoire, processeurs, réseau, ...).

Environnement d'exécution des logiciels

Fournir un environnement d'exécution aux applications qui « tournent » sur la machine (temps processeur, mémoire, accès aux périphériques) sans qu'elles se gênent. Exemples :

Environnement d'exécution des logiciels

Fournir un environnement d'exécution aux applications qui « tournent » sur la machine (temps processeur, mémoire, accès aux périphériques) sans qu'elles se gênent. Exemples :

- empêcher une application d'écrire dans la mémoire d'une autre application (ou pire du noyau) ;
- empêcher une application de monopoliser les ressources de calcul alors que d'autres processus attendent ;
- empêcher l'application d'un utilisateur lambda de modifier les fichiers du système sur le disque dur.

Noyau comme service

Le noyau maintient pour le programmeur et l'utilisateur un petit nombre d'abstractions de manière à cacher les détails techniques et à rendre les applications portables. Sous Unix, les deux abstractions principales sont :

Noyau comme service

Le noyau maintient pour le programmeur et l'utilisateur un petit nombre d'abstractions de manière à cacher les détails techniques et à rendre les applications portables. Sous Unix, les deux abstractions principales sont :

- le processus, programme chargé en mémoire, qui a une vie ;
- le fichier, toute chose dans lequel les processus peuvent lire et/ou écrire des données, qui a une place.

Les services rendus par le noyau permettent de s'extraire des détails internes et sont accessibles *via* une API standardisée (POSIX). Ainsi, par exemple, une application peut écrire des données dans un fichier sans se demander s'il est stocké sur un disque SSD, un disque HDD ou une clé USB.

Contraintes et difficultés

- performance
- fiabilité
- sécurité
- portabilité : fonctionner avec des architectures différentes (par exemple x86-64, les différentes architectures ARM...)
- capacité d'interagir avec des périphériques toujours plus nombreux
- gérer la complexité toujours plus grande des machines, en particulier depuis que la fréquence d'horloge des processeurs a cessé de croître très rapidement.
- Toutes ces contraintes font des noyaux des programmes parmi les plus difficiles à écrire et maintenir.

Appels système

- Le noyau a le monopole de l'accès au matériel.
- Lorsqu'une application a besoin d'une ressource (par exemple, demander plus de mémoire, demander à lire ce que tape l'utilisateur au clavier, ou à écrire des données sur le disque dur), elle exécute une instruction appelée appel système qui :
  - fait passer le CPU en mode *privilégié* (*sys*)
  - saute à l'adresse correspondant à cette demande dans la partie « noyau » de la RAM.
- Dans la partie noyau :
  - décision sur le fait que l'application peut ou non accéder à cette ressource ;
  - vérification que cette ressource est disponible ;
  - éventuellement mise en sommeil du processus jusqu'à la disponibilité de la ressource ;
  - si pas d'erreur, accès à la ressource et retour à l'application en mode non privilégié (*user*) ;
  - si erreur, retour à l'application avec informations sur l'erreur.

Appels système, résumé et exemples

En résumé, un appel système est une instruction du programme qui fait un saut dans la partie noyau de la mémoire pour accéder à un service dont le noyau est responsable. Par exemple :

- créer un processus : `fork`
- tuer un processus `exit` (soi-même), `kill` (un autre)
- changer la taille de la mémoire allouée à un processus : `brk`
- charger un programme en mémoire : famille `exec`
- ouvrir/fermer un fichier : `open` et `close`
- lire/écrire dans un fichier : `read` et `write`

Ces appels sont

- soit disponibles tels quels en C, dans ce cas, section 2 du manuel (appels systèmes), par exemple ; `man 2 read`
- soit enveloppés dans une fonction d'une bibliothèque, dans ce cas section 3 du manuel (fonctions des bibliothèques C), par exemple `man 3 execlp`.

Appels système : `hello, world!`

Le programme `strace` permet de visualiser les appels systèmes que passe un processus (exemple sous Linux 5.15).

```
$ gcc -Wall hello.c
$ strace ./a.out
execve("./a.out", ["./a.out"], 0x7ffcc04ed280 /* 63 vars */) = 0
brk(NULL)                                   = 0x108c000
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
(...)
write(1, "hello, world!\n", 14hello, world!
)          = 14
exit_group(0)                               = ?
+++ exited with 0 +++
```

Appels système : `hello, world!`

Le programme `strace` permet de visualiser les appels systèmes que passe un processus (exemple sous Linux 5.15).

```
$ gcc -Wall hello.c
$ strace ./a.out
execve("./a.out", ["./a.out"], 0x7ffcc04ed280 /* 63 vars */) = 0
brk(NULL)                                   = 0x108c000
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
(...)
write(1, "hello, world!\n", 14hello, world!
)          = 14
exit_group(0)                               = ?
+++ exited with 0 +++
```

Ça fait beaucoup et Linux a ses propres appels utilisés dans les fonctions bas niveau *portables* des bibliothèques.
(Liste de tous les appels système : `man -k . -s 2`).

Noyau et appels système

Processus Unix

Rappels de C : `switch`

Processus : naissance et mort (1)

Processus : *être*

Définition la plus courante (par exemple Maurice J. Bach, “The design of the Unix operating system”) :

> *Un programme est un fichier exécutable et un processus est une instance de ce programme en cours d’exécution.*

- Différence entre le *programme*, **inerte** et le *processus*, exécution de ce programme, **dynamique**.
- Instance : anglicisme dur à traduire, disons « cas particulier avec un état initial ou des paramètres donnés »

Tous les systèmes d’exploitation modernes utilisent, d’une façon ou d’une autre, l’abstraction « processus ».

Processus Unix

- Unix est, dans son ADN, multi-processus car dès le départ multi-utilisateurs, à temps partagé (*time sharing*)
- L'ordonnanceur (*scheduler*) du système partage le temps de calcul du ou des processeurs entre les processus. Grossièrement (plus de détails dans un prochain cours) :
  - les processus sont placés dans une file d'attente ;
  - celui qui est en tête est *élu* : utilise le processeur pendant une courte période (0.1 seconde, par exemple)
  - s'il n'est pas terminé, retourne au bout de la file.
  - Algorithme *round-robin* ou *tourniquet*.
- Gestion des processus en attente d'événement : les endormir et les réveiller le moment venus.
- Chaque processus a son propre *espace mémoire*.
- Un processus peut avoir plusieurs *fils*[1] *d'exécutions* (*threads*) qui sont aussi ordonnancés par le noyau.

1. comme un fil pour coudre

PID et état d'un processus

- Chaque processus est identifié par un entier > 0 : son `PID` (pour *Process Identifier*).
- Les permissions d'un processus sont celles de son propriétaire utilisateur et de son groupe propriétaire.
- Un processus a un état, les plus courants :
  - `S` : pour *sleeping*, en attente d'un événement (entrée utilisateur, fin d'un autre processus, ...)
  - `R` : pour *running*, les instructions du programme sont exécutées par un ou plusieurs processeurs, ou bien le processus est dans la file d'attente
  - `T` : interrompu (mis en pause) par un signal
  - `Z` : Zombie, mort mais non libéré car son parent est vivant et n'a pas récupéré son statut de sortie.
- Liste de tous les processus : `ps -elf` (syntaxe Unix) ou `ps aux` (syntaxe BSD).

Chercher un processus

Il y a plusieurs façons de chercher un processus (c'est-à-dire son PID) :

- PID du shell courant : `$$`
- PID du dernier processus lancé en arrière-plan : `$!`
- PID de tous les processus lancés en arrière-plan : `jobs -p`
- PID du processus lancé par la commande de nom de commande `cmd` : `ps -C cmd`
- Le bon vieux `ps aux | grep cmd` (*pro tip :* `ps aux | grep [c]md` pour éviter que la commande `grep` elle-même apparaisse).

Et après ?

- On peut le tuer : `kill <pid>`
- On peut examiner ses attributs plus finement :
  - `ps -lf -p <pid>`
  - `ps -p <pid> -o ppid,tty,s,uid` (ou tout autre choix d'attributs)
- On peut aller voir (sur Linux) le contenu de `/proc/<pid>`, exemples :
  - `cat /proc/<pid>/environ | tr '\000' '\n'` afficher l'environnement du processus
  - `ls -l /proc/<pid>/fd` afficher les fichiers ouverts et leurs descripteurs
  - `cat /proc/<pid>/cmdline | tr '\000' '\n'` afficher la ligne de commande qui a lancé le processus
  - `ls -ld /proc/<pid>/cwd` afficher le répertoire courant du processus
  - *etc*

Noyau et appels système

Processus Unix

Rappels de C : `switch`

Processus : naissance et mort (1)

switch : Exemple

```
#include <stdio.h>

int main(void)
{
    char c;

    while (scanf(" %c", &c) != 0) {
        switch (c) {
        case 'a':
            printf("lol\n");
        case 'b':
            printf("hop\n");
            break;
        case 'Z':
            return 0;
        default:
            printf("plop\n");
        }
    }
}
```

Exécution

```
$ gcc -Wall exemple_switch.c
$ ./a.out
a
lol
hop
b
hop
r
plop
Z
```

`switch` : récapitulatif

`switch` correspond à un traitement *par cas*

- sur une expression *entière*, qui est *évaluée* ;
- l'instruction saute à la première étiquette `case TRUC:`, où `TRUC` est une expression entière constante, *connue à la compilation*, égale à la valeur de l'expression, s'il y en a ;
- puis toutes les instructions contenues dans le `switch` sont exécutées ;
- jusqu'à une éventuelle instruction `break`, ou bien sûr toute autre instruction provoquant un saut en-dehors du `switch` (par exemple `return` depuis une fonction).
- Une éventuelle étiquette `default:` attrape toutes les valeurs de l'expression.

switch : autre exemple

Un cas peut être vide.

```
int est_bissextile(int annee)
{
    return (annee % 400 == 0) ||
           (annee % 4 == 0 && annee % 100 != 0);
}
int nb_jours_mois(int mois, int annee)
{
    switch (mois) {
    case 4: case 6: case 9: case 11:
        return 30;
    case 2:
        return 28 + est_bissextile(annee);
    default:
        return 31;
    }
}
```

Noyau et appels système

Processus Unix

Rappels de C : `switch`

Processus : naissance et mort (1)

Arborescence des processus

- Seul un processus peut créer un nouveau processus.
- À part, évidemment, le tout premier processus, celui de `PID` 1 qui est créé au démarrage du système :
  - `init` sur les Unix traditionnels ;
  - `systemd` sur la plupart des systèmes GNU/Linux aujourd'hui ;
  - `launchd` sur MacOS X.
- Lorsqu'un processus demande au noyau la création d'un nouveau processus et que celle-ci est acceptée, on dit qu'il *fork*.
  - Ce nouveau processus *naît* et le processus qui l'a créé est son *parent*.
  - Le nouveau processus est *l'enfant* de celui qui l'a créé.
- Ainsi, les processus créent une *arborescence* dont la racine est le processus de `PID 1`, qui est l'ancêtre commun de tous les processus (si elle est installée, la commande non standard `pstree` montre cette arborescence).
- Le `PID` du processus parent s'appelle le `PPID` (*parent process id*).

fork

La création d'un processus se fait par l'appel système

```
#include <unistd> /* fork */
#include <sys/types.h> /* type pid_t */
pid_t fork(void);
```

- Crée une *copie* (presque) conforme du processus appelant.
- À partir du `fork`, les deux processus vivent leurs vies de façon indépendante.
- L'une des seules différences est la valeur de retour de `fork` :
  - le processus qui a reçu 0 est l'enfant ;
  - le processus qui a reçu un entier > 0 est le parent, l'entier reçu est le `PID` de cet enfant.
- En cas (rare) d'erreur, `fork` retourne -1. Arrive si la table des processus est pleine (par exemple si on lance une *fork bomb*).
- Le type `pid_t` est un type entier signé (pour pouvoir contenir -1), qui dépend du système (souvent c'est `int`).

Exemple (pédagogique) : exemple_fork.c

```c
#include <unistd.h>
#include <sys/types.h>
#include <stdio.h>
int main(int argc, char *argv[])
{
    int a = 3, b = 5;
    pid_t p;
    switch ((p = fork())) {
    case -1:
        perror("fork");
        return 1;
    case 0:
        printf("%d: je suis l'enfant de %d\n", getpid(), getppid());
        b = 42;
        break;
    default:
        printf("%d: mon enfant est %d\n", getpid(), p);
        a = -2;
    }
    printf("%d: a = %d, b = %d\n", getpid(), a, b);
    return 0;
}
```

Sortie et commentaires

```
$ ./a.out
41928: mon enfant est 41929
41929: je suis l'enfant de 41928
41928: a = -2, b = 5
41929: a = 3, b = 42
```

- Les deux processus exécutent le dernier `printf` et l'instruction `return 0`.
- La mémoire est bien copiée : la pile du parent contenant les variables `a` et `b` a bien été recopiée chez l'enfants ;
- mais une modification faite par un processus reste locale à celui-ci (le `b` du parent contient encore `5` et le `a` de l'enfant `3`).
- On a vu aussi les appels système :

```
pid_t getpid();
pid_t getppid(); /* Parent PID : PID du parent. */
```

fork bomb

```c
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

int main()
{
    switch (fork()) {
    case -1:
        perror("fork");
        return 1;
    case 0:
        for (;;)
            if (fork() == -1)
                perror("fork");
    }
    /* pour ne pas tout planter */
    sleep(5);
    kill(0, SIGTERM);
    return 0;
}
```